Retrieval-Augmented Generation (RAG): Grounding, Knowledge Sources, Chunking and Embeddings

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain the three steps of RAG (retrieval, augmentation and generation) using the open-book exam analogy
- Compare how an LLM answers a question with and without RAG and grounding
- Identify common knowledge sources and split a document into context-driven chunks in Python
- Describe how embeddings and semantic similarity help a RAG system find the right chunk

2. Detailed Explanation

A quick warm-up: what "augmented" means

Before we unpack RAG, let's borrow a word you have probably seen before. The "A" in RAG stands for "augmented", and two familiar terms help set the scene.

- Virtual reality (VR) is a reality in a virtual format. You are not physically present somewhere, but you are present virtually, like when you wear 3D goggles for gaming.

- Augmented reality (AR) places digital elements as 3D models into your real view. Pokemon Go is a classic example. You scan your surroundings with a mobile phone, and an object appears in front of you in 3D.

Keep that idea of "adding something on top of what is already there" in mind. It is exactly what the middle step of RAG does.

What RAG stands for

RAG stands for Retrieval-Augmented Generation. Each word names one step of the process.

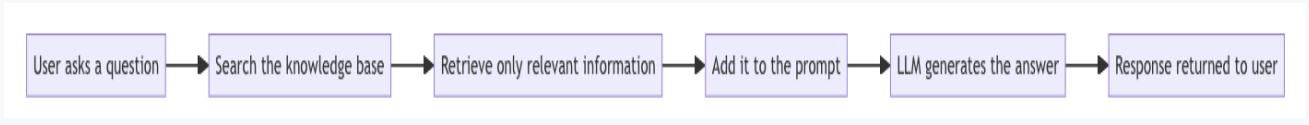
Word	What it means
Retrieval	Fetching or pulling out the *relevant* information
Augmented	Adding that retrieved information to the prompt
Generation	Asking the AI model to generate an answer using the provided information

Put together, RAG is a technique in which an AI application retrieves relevant information from a specific knowledge source. It then gives that information to the large language model (LLM), which uses it as context to generate the outcome.

How the RAG process works

All of these steps run quietly in the background of a RAG-based application. Here is the sequence from question to answer.

1. A user asks a question.
2. The question passes through the RAG layer, which retrieves relevant information.
3. The retrieved information is added to the prompt. This is augmentation.
4. The augmented prompt is passed to the LLM. This is a generation.
5. The LLM generates the response.



The knowledge base is simply the data available to the system. Notice the word "relevant" in the flow. Not everything in the knowledge base helps with a given question, so the system pulls out only the parts that do.

The open-book exam analogy. Picture a student in an open-book exam who cannot remember an answer. They are allowed to use their notes or textbook. They do not flip pages at random. They use the index to jump to the topic, find the relevant page, read it, and write the answer in their own words.

- Finding the relevant page is retrieval.
- Using what you read on that page is augmentation.
- Writing the answer is generation.

RAG is an add-on, not a requirement

Here is something that surprises many beginners. An LLM does not need RAG to work. RAG is a layer of improvement added on top of a model.

So why add it? Compare the two situations side by side.

	LLM without RAG	LLM with RAG
Where the answer comes from	Information the model already learned	A supplied document, such as the latest company policy
Example question	"What is the registration process for this course?"	Same question, or one about refunds
What happens	The model answers from existing knowledge, which may be old	The system searches the policy document, finds the relevant part and sends it to the LLM
Risk	The model may have to guess, which can lead to an incorrect answer	The answer is based on the current document

Watch out for: assuming an LLM without RAG is always wrong. It does not always produce a wrong answer. The point is that the risk of a guess based on old knowledge is there.

Grounding: tying answers to a trusted source

Grounding means generating an AI response based on relevant, trusted information that a specific source provides to the LLM. If a company policy document is the trusted source, the answer is said to be grounded in the retrieved information.

The difference between the two paths comes down to one step.

- Without grounding: the question goes to the LLM, the LLM gives a possible answer, and the process ends.
- With grounding: the question is paired with relevant information from a specific, trustworthy source. That information goes to the LLM, which then generates the response.

You need grounding wherever the answer depends on a specific source rather than general knowledge. Medical responses are one example. Grounding helps most with tasks that need:

- Company-specific information
- Domain-specific information
- Product-specific information

Knowledge sources

A knowledge source is where the information supplied to the LLM comes from. Think of it as the "textbook" in the open-book exam. Common types include:

- Documents: PDF, Word, plain text and Markdown files, such as company policy documents
- Structured databases: Excel files, CSV files and SQL databases
- Internal systems and tools: data fetched from CRM software or ERP software
- Websites: the company's official website and its documentation
- FAQs: pages of frequently asked questions

In most systems, access to these sources requires the user's permission.

Sometimes a knowledge source is not just helpful but mandatory. Take the question "Which laptop products are currently available?" Without the company's list of available

products, the model has no way to work out the answer. The question is baseless without the company's own database or documentation.

Chunking: breaking big documents into useful pieces

Imagine a company policy document that runs to 100 or 200 pages or more. You could send the whole thing with every user question, and it would technically work because the model does need the information. It is still not practical, for three reasons:

- It is inefficient to pass that much information every time.
- It can exceed the context limit of the model.
- It is costly, because every query needs a large amount of processing.

The fix is chunking, which means breaking a document into smaller pieces called chunks.

This follows the same divide-and-conquer idea used in teaching. A six-month course is not delivered all at once. It is divided into smaller parts and taught week by week, chapter by chapter. Chunking does the same thing to a document: divide it into smaller chunks and conquer each one.

Watch out for: treating chunking as simple splitting into equal pieces. Good chunking is context-driven, so each chunk keeps related information together. A company leave policy shows why. It may contain several distinct segments:

- Employees receive a fixed number of paid leaves per year (for example, 15 or 18).
- A leave must be applied for five working days in advance.
- Emergency leave can be requested without notice, but only on rare occasions.

Each segment can become its own chunk, instead of the policy sitting as one giant block.

Now see the payoff. Suppose the leave policy is 500 pages and someone asks, "How many paid leaves will I get in a year?" Without chunking, the model has to scan the entire document, even though the answer sits in one small section. With chunking, the search runs only over the relevant chunks. That makes a large difference in efficiency.

Hands-on: chunking text in Python

Let's try a small exercise. The text below has sentences separated by two line breaks, as if someone pressed Enter twice between them. The goal is to make each sentence a separate chunk stored in a list.

The key decision is the delimiter, the character pattern you split on. Here is how the options compare:

- A space is not correct.
- A single is not correct. It does not match the two line breaks.
- A full stop seems reasonable, but here it leaves unwanted line-break characters in the chunks. It would only be right if sentences were separated by full stops with no line breaks.
- `

` is correct, because it matches the two line breaks exactly.

```
text = """Employees receive 18 paid leaves per year.

Employees must apply for leave five working days in advance.

Emergency leave may be requested without prior notice."""

sentences = text.split("

")
```

```
i = 1
for s in sentences:
    print("Chunk number", i, s)
    i += 1
```

`text.split("

")`returns a list called` sentences`, one element per chunk. The loop prints each chunk with the counter` i`. The output looks like this:

```
Chunk number 1 Employees receive 18 paid leaves per year.
Chunk number 2 Employees must apply for leave five working days in advance.
Chunk number 3 Emergency leave may be requested without prior notice.
```

Real text will not always use double line breaks. It might follow another pattern, such as full-stop separation, and your chunking then depends on that pattern. Sometimes you need pre-processing first, such as removing unnecessary spaces, before the text is ready to be chunked.

Embeddings and semantic similarity

Once you have chunks, the system still needs a way to tell which chunk matches a question. That is where embeddings come in.

An embedding represents text in a numerical format so that a computer can extract meaning from it. This is needed because a computer only understands numerical data. An embedding model converts any sentence into a numerical vector, with values such as 0.23 (the numbers in this illustration are arbitrary). You cannot make sense of these numbers just by looking at them. However, each vector is a point in a vector space, and similar words end up closer to each other in that space.

Semantic similarity means that words or sentences with similar meanings lie close together in the vector space. Embeddings are what make semantic similarity possible.

Here is the core use case in RAG. A policy document contains the sentence "Customers can return a product within 30 days." A user then asks a question whose answer lies in that sentence.

- Without semantic similarity, which depends on understanding meaning, the system cannot recognise that the question and the sentence are linked.
- With embeddings, the meaning of both is compared, and the matching sentence is found.

How the pieces fit together

Each idea in this lesson builds on the one before it:

1. The name RAG splits into retrieval, augmentation and generation.
2. The open-book exam shows those same three steps in everyday terms.
3. An LLM without RAG risks old information and guesses, so RAG is an add-on improvement.
4. Grounding ties answers to a trusted, specific source.
5. Knowledge sources supply that trusted information.
6. Large sources are too costly to send whole, so they are broken into chunks.
7. Embeddings turn chunks and questions into vectors, so semantic similarity can find the relevant chunk.

3. Key Takeaways

- RAG means retrieving relevant information, adding it to the prompt, and letting the LLM generate an answer from it.

- RAG is an optional layer on top of an LLM. It reduces the risk of answers based on old or missing knowledge.
- Grounding ties a response to a trusted, specific source, drawn from knowledge sources like documents, databases, internal systems, websites and FAQs.
- Chunking breaks large documents into context-driven pieces so the system searches only what is relevant, which saves cost and stays within the context limit.
- Embeddings turn text into vectors, and semantic similarity lets the system match a question to the chunk with the same meaning.

Mental model: Think of RAG as an open-book exam. The student uses the index to find the right page (retrieval), reads it (augmentation), and writes the answer in their own words (generation).